Constructor, the names are automatically generated. A thread created using the Constructor above would be named:

Thread-0, Thread-1, Thread-2, etc.

To instantiate a Thread using this default Constructor, we would do like this:

Thread t1 = new Thread();

t1 would have the thread name of Thread-0.

2.Thread(String name)

This Constructor takes a String argument, which becomes the thread's name.

```
String n = "cece";
Thread t2 = new Thread( n );
```

t2 would have the thread name of cece.

Similarly,

```
String n = "cece";
Thread t3 = new Thread( n );
Thread t4 = new Thread();
```

t3 would have the thread name of cece.

t4 would have the thread name of Thread-0.

(There are 5 more Constructors with more ambitious intentions. For now, these two are sufficient.)

Once we have instantiated a thread object, we'd like to put it to work. All of the work done by a thread happens in the method:

run()

This method takes no arguments and returns void.

Normally, you will never call the run() method yourself. You will call another method— start() —and the JVM will call run() when it is good and ready.

As you would expect, we begin by extending the class Thread.

```
public class MyThread extends Thread
{
}
```

First, add the constructor method—which names the thread "Thread-x". Now, because it sub-classes Thread, class MyThread is an instance of the thread class. Class Thread contains a method run() that is empty—it does nothing. So, if you want anything to happen in your thread, you need to